TARUMT
TUNKU ABDUL RAHMAN UNIVERSITY OF MANAGEMENT AND TECHNOLOGY

BACS3403 PROJECT 1
FORM 2 : PROPOSAL

Student Name	**POONG FOO JING**		
Programme	**Bachelor in Information Technology (Honours) (Software Systems Development)**		
Email	**poongfj-sm23@student.tarc.edu.my**	**Mobile Phone No**	**012-508 1025**
Supervisor Name	**Mr. LIM JIA ZHENG**		

PROJECT TITLE

TARUMT Class Replacement System

ABSTRACT

The current manual class replacement process at TAR UMT Sabah relies on static Google Sheets, causing significant administrative delays, operational inefficiencies, and recurring scheduling conflicts among lecturers, multi-cohort student groups, and venue bookings. To resolve this, this project proposes the TARUMT Class Replacement System. Developed within a tight 7-week phase using a Laravel and PostgreSQL framework, the web-based system replaces fragmented spreadsheets with an automated multi-entity intersection algorithm. It performs a cross-check analysis based on live schedule involving lecturers' calendar, room status, and multi-cohorts student schedule and programmatically generates the list of absolute common free slots. Lecturers can then choose their verified slot after consultation with students, and make use of optimistic concurrency control to prevent double bookings. Finally, the request will be routed to First-Come-First-Served Dashboard for Programme Leader's quick approval.

PROBLEM

The management of replacement classes at the TAR UMT Sabah currently involves a decentralised web of Google sheets. There are five major interrelated weaknesses inherent in this system.

Fragmented Process and Human Errors

The management of replacement requests manually between different instances of spreadsheet involves numerous cross-referencing efforts on behalf of the academic staff members. Every lecturer keeps his or her own schedule, while there is no database available to unify the state of replacements in the faculty. Several studies proved that manual scheduling process in educational organizations increases the possibility of human mistakes (Babaei, Karimpour, & Hadidi, 2015).

Blind Spots in Multi-Cohort Scheduling

In many cases, one module will have students belonging to several groups of academic year, like RSD3-S1 and RSD3-S2. Manually combining schedules of two or more different cohorts, in order to find common free periods, will take much time and can result in mistakes especially due to the fact that these cohorts usually study different modules.

Venue Availability Tracking

Information regarding the venue availability is stored separately from the scheduling process. It

means that lecturers have to independently check whether classrooms are free for replacement or not.

Data Race Conditions

The collaborative nature of the shared Google Sheets workspace fails to offer any kind of transaction boundaries. In case multiple lecturers are trying to make their requests at the exact same time, both will see the same slot free, creating an unsolvable conflict of double booking. Collaborative spreadsheet applications in the context of multi-user scheduling have already been recognized by previous research (Al-Hawari, Al-Sadi, & Barham, 2021).

Increased Administrative Effort for Programme Leaders

Programme Leaders (PLs) have to manually check each new request, verify it against schedules of the lecturers and cohorts, and confirm availability of the venues independently.

SOLUTION

This solution proposes a centralised web-based scheduling application that would substitute the currently used spreadsheet approach by an algorithmic intersection engine. It works as a single platform that accepts input from various sources regarding timetable data, finds valid replacement windows via set intersection, and provides transactional integrity via Optimistic Concurrency Control (OCC).

The system solves the problems outlined above by:

- Substituting the manual cross-checking process of lecturer/cohort timetables by the automatic matrix intersection engine capable of considering all of the constraints at once.
- Eliminating the possibility of double booking via the Optimistically Controlled transactional layer that checks the validity of transactions at the time of their execution.
- Providing a native set intersection capability, which makes it possible to handle mixed-cohort groupings.
- Reducing the administrative load on Programme Leaders due to a centralised First-Come-First-Served digital approval dashboard.

Critical Functions:

Multi-Entity Matrix Intersection Engine

The back end uses a deterministic algorithm of set intersection, working with four vectors simultaneously: lecturer temporal availability vector, cohort free schedule vectors (handling mixed-cohort groupings), room occupancy vector, and venue filtering based on capacity awareness that excludes venues with the number of seats less than the total number of students in affected cohorts are filtered out.

Layer for Optimistic Concurrency Control (OCC)

The system executes a validation query in isolation during that millisecond exactly when the database is about to be updated. If the transaction carried out by any other user has changed the slot state during those milliseconds, then the system ends that transaction, rolls back all updates in the transaction, and gives an alert in the UI to ask the user to refresh their screen.

Visual Indicators of State of Slots

All the candidate slots for replacement along with their transaction states are indicated through the user interface of the lecturer. The states include the following:

- **Available** – The slot is neither reserved nor any constraint violation exists.
- **Pending (Self)** – The replacement request made by the currently authenticated user is pending approval from the Programme Leader (PL).
- **Reserved by Others** – The slot is reserved by some other user who has more priority compared to the currently authenticated user according to First-Come-First-Serve principle.
- **Occupied** – The slot is occupied by some other teaching task and hence cannot be replaced.

First-Come-First-Served (FCFS) Approval Interface

The programme leaders are presented with a chronologically ordered queue of all submitted requests. The interface enables one-click approval of request (slot is put in an Occupied state) or disapproval of request (with required textual explanation, slot becomes Available).

Role Inheritance for Programme Leaders in Hybrid Form

Both appointed Programme Leaders have complete lecturer level creation and submission rights, giving them the ability to make replacement requests for their respective modules.

Role-Based Access Control

Students have only viewing access to the system. The lecturers can create new replacement requests and submit them. Programme Leaders have administrative queue management functionality in addition to what lecturers have.

EXPLAIN HOW THE PROJECT CONTRIBUTED TO THE SELECTED SDGs

SDG 4 : Quality Education

Explanation/Justification

Any disruptions to classroom schedules due to non-availability of lecturers, availability of rooms, or other reasons interfere with learning activities. By reducing the time for scheduling replacement from several days to seconds through automation, the system enables continuous teaching process without any disruption to the academic calendar.

SDG 8 : Decent Work and Economic Growth

Explanation/Justification

By automating administrative coordination, academic staff members can be relieved of spreadsheet management, allowing them to focus on teaching activities and student mentoring. Besides, the Optimistic Concurrency Control (OCC) system structure can be considered a good example of integrity in an educational resource management system.

TARGET MARKET

Lecturers (Department of Computing and Information Technology, TAR UMT Sabah)

The primary users of the system are the faculty members who teach various modules among different cohorts in the faculty. The system helps the users in starting the process of requesting replacements through selecting the relevant class block requiring rescheduling, using the matrix intersection algorithm to produce the possible replacement slots, checking the availability status and selecting the preferable time and venue in consultation with the students, and sending out the replacement request to the Programme Leaders for approval. There will be about five lecturers who will use the system as part of the prototype and all belong to the DCIT department.

Programme Leaders (DCIT)

Two users who have both administrative and teaching positions in the faculty. Programme Leaders use the system to access the FCFS approval dashboard which shows the submitted requests chronologically, check the requests along with the slot validity results produced by the system, approve the valid request by clicking on it which sets the slot into the Occupied state forever, reject any request that is invalid or unsuitable by providing the necessary reason which resets the slot into the Available state, and carry out all other lecturers' activities like making the replacement request.

Students (Multi-Cohort Groups)

The ultimate beneficiaries of the entire system are the students registered at the FOCS faculty. The students log into the system using valid credentials, but they only enjoy read-only permission. The students make use of the system to check on their consolidated master timetable indicating synchronised classes, to get real-time information on how a replacement request is approved and changes made to the schedule, and to monitor the status of any replacement request affecting the courses they are doing.

COMPETITION / CONTRIBUTION

Comparison to Existing Solutions

The existing solution based on Google Sheets has some drawbacks which the suggested system solves. Spreadsheets have no transactions, thus multiple user modifications may cause losses due to double booking. Cross-cohort schedule comparisons should be done manually through overlapping different files, which is inefficient and prone to mistakes.

The proposed system uses Optimistic Concurrency Control (OCC), a design approach that helps maintain good performance when multiple users access the system at the same time without using database locks. This is a suitable choice for a faculty environment, where replacement slots are sometimes requested by several users, while the impact of double booking is significant. The Multi-Entity Matrix Intersection Engine also supports multiple cohorts, allowing the system to calculate valid replacement slots for classes involving more than one cohort.

Contribution to Stakeholders

The system makes contributions to the FOCS faculty members in three primary ways. First, it prevents data races resulting from double bookings by using OCC. Second, it implements the cross cohort scheduling through a matrix intersection engine which automatically calculates the correct replacement window times for groups with multiple cohorts. Third, it creates an audit trail of all

replacement transactions for Programme Leaders in chronological order.

There is also a possibility of presenting this solution at technology conferences of the institution such as TAR UMT Smart Campus Initiative and using it as a reference solution for concurrency controlled scheduling in similar academic environments.

MILESTONES

Proposed Development Model / Research Method : **Agile (Iterative Approach)**

Due to the limited development period, the Agile development method will be adopted. This approach supports rapid development through short and repeated improvement cycles, allowing early testing of important system components such as the database structure and core functions. High-risk features, including schedule matching and double-booking prevention, will be prioritized during the early stages of development. This helps identify issues quickly and ensures that the main system functions correctly before deployment.

Project Schedule:

ACTIVITIES	EXPECTED OUTCOME	COMPLETION DATE
Requirements Elicitation and Problem Formulation	Approved Form 2 Proposal, refined project scope	FYP1 Week 2
Chapter 1: Introduction	Completed Chapter 1 with problem statement, objectives, and project scope	FYP1 Week 5
System Requirements and Use Case Modelling	Software Requirements Specification (SRS), use case diagrams	FYP1 Week 6
Chapter 2: Literature Review	Completed Chapter 2 with APA-cited review of OCC, timetabling algorithms, and related systems	FYP1 Week 8
Relational Database Schema Design	Normalised PostgreSQL schema, Entity Relationship Diagram	FYP1 Week 9
Chapter 3: Methodology and Requirements Analysis	Completed Chapter 3 with methodology, functional and non-functional requirements	FYP1 Week 10

Chapter 4: System Design	Completed Chapter 4 with system architecture, ERD, UI mockups, OCC pseudocode	FYP1 Week 12
Interim Prototype and Core Logic Setup	Application routing framework, preliminary seed data ingestion	FYP1 Week 13
FYP1 Portfolio Submission	Compiled Chapters 1–4 report, interim viva presentation	FYP1 Week 14
Sprint 1: Matrix Intersection Engine	Functional core API for deterministic slot intersection	FYP2 Week 2
Sprint 2: OCC Layer and FCFS Dashboard	Conflict prevention logic, administrative approval queue	FYP2 Week 4
Sprint 3: UI Integration and System Verification	Complete web interface, test case report, UAT sign-off	FYP2 Week 5
Final Thesis Compilation	Completed FYP2 report (Chapters 5–7), system documentation	FYP2 Week 6
Final Viva Defence and Submission	Formal presentation, live demo, final report submission	FYP2 Week 7

REFERENCES

1. Ghamdi, H., Alsubait, T., Alhakami, H., & Baz, A. (2020). A review of optimization algorithms for university timetable scheduling. Engineering, Technology & Applied Science Research, 10(6), 6410–6417. https://doi.org/10.48084/etasr.3832

2. Babaei, H., Karimpour, J., & Hadidi, A. (2015). A survey of approaches for university course timetabling problem. Computers & Industrial Engineering, 86, 43–59. https://doi.org/10.1016/j.cie.2014.11.010

3. Bernstein, P. A., Hadzilacos, V., & Goodman, N. (1987). Concurrency control and recovery in database systems. Addison-Wesley. https://www.microsoft.com/en-us/research/people/philbe/book/

4. Herlihy, M., & Wing, J. M. (1990). Linearizability: A correctness condition for concurrent objects. ACM Transactions on Programming Languages and Systems, 12(3), 463–492.

https://doi.org/10.1145/78969.78972

5. Kung, H. T., & Robinson, J. T. (1981). On optimistic methods for concurrency control. ACM Transactions on Database Systems, 6(2), 213–226. https://doi.org/10.1145/319566.319567

6. Schaerf, A. (1999). A survey of automated timetabling. Artificial Intelligence Review, 13(2), 87–127. https://doi.org/10.1023/A:1006576209967A

7. Al-Hawari, F., Al-Ashi, M., Abawi, F., & Alouneh, S. (2020). A practical three-phase ILP approach for solving the examination timetabling problem. International Transactions in Operational Research, 27(2), 924–944. https://doi.org/10.1111/itor.12471